

CC -9 : VB.NET

Unit 1 : Introduction – .NET Framework Technology, Advantage, Framework Class Library :data types, exceptions, events, attributes, collections, remote calls, threading, web services etc., Working with .NET FCL : namespaces.

Unit 2 : Language – Data Types, Operators, Modifiers, Directives, Programming Constructs: decision making, case and iteration, Arrays, Procedures & Functions, Library Functions: string, date & time, math, number and Miscellaneous.

Unit 3 : OOPS – Features, Declaring classes, Properties, Public & Private variables, Inheritance, Overloading, Overriding, Virtual Methods and Abstract Class.

Unit 4 : Windows Form – Introduction to Class Libraries, Event and Event Handlers, Winforms and GUI, Creating Applications using Winforms, Concept of Controls, Different GUI Controls, their properties, methods and event procedures and Programs using controls.

Unit 5 : Error Handling in Windows Form – Introduction to Errors, Exceptions & Validations, Types of Validations, Types of Errors, Types of Exceptions.

Unit 6 : SDI & MDI Applications – Introduction, Characteristics & Drawbacks, File Handling program.

Unit 7 : Data Access – Introduction to Microsoft Data Access Technologies, ADO.NET : overview, architecture, class libraries, Creating Database, Connecting to Database, Databound Controls, Creating Data Set, Manipulating Records and XML Data.

Unit 8 : Component Object Module (COM) – Fundamentals of Custom Control, Introduction to COM, Events and User Controls, Web Programming.

Unit 1: Introduction to .NET Framework Technology

1. Overview of .NET Framework Technology

The .NET Framework is a software development platform developed by Microsoft. It provides a controlled programming environment where software can be developed, executed, and maintained. At its core, the framework was designed to address classic software development challenges, such as platform dependency, memory management burdens, and the notorious "DLL Hell" (where installing one application overwrites shared libraries required by another).

The "Why" Behind .NET

Historically, applications were compiled directly to machine code specific to a target operating system and hardware architecture. Developers had to manually manage memory allocation and deallocation, leading to memory leaks and security vulnerabilities. The .NET

Framework introduced a managed execution environment. The primary real-world purpose of .NET is to provide hardware and language independence, allowing developers to write code in various languages (C#, VB.NET, F#) that seamlessly interoperate and execute securely on a unified runtime.

Core Architecture Execution Process

To understand how .NET achieves language interoperability and platform independence, the execution of a .NET application must be broken down into distinct phases:

1. **Language Compilation:** Source code written in any .NET-compliant language is compiled by its respective language compiler (e.g., the C# compiler).
2. **Translation to Intermediate Language:** Instead of compiling directly to native machine code, the compiler translates the source code into Common Intermediate Language (CIL), previously known as MSIL. Simultaneously, the compiler generates Metadata, which describes the types, members, and references in the code.
3. **Assembly Generation:** The CIL and Metadata are packaged together into a portable executable file called an Assembly (either an .exe or a .dll).
4. **Just-In-Time (JIT) Compilation:** When the application is executed, the Common Language Runtime (CLR) activates. The CLR's JIT compiler reads the CIL and translates it on demand (method by method) into the native machine instructions of the host underlying hardware.
5. **Managed Execution:** The CPU executes the native machine code while the CLR continuously provides core services such as garbage collection, exception handling, and thread management.

2. Advantages of the .NET Framework

Utilizing the .NET Framework provides distinct architectural and operational benefits for enterprise software development, though it also introduces certain structural trade-offs.

Advantages

- **Consistent Programming Model:** Access to files, databases, networks, and user interfaces relies on a standardized object-oriented API, reducing the learning curve across different programming domains.
- **Language Interoperability:** Governed by the Common Language Specification (CLS), components written in C# can inherit from classes written in VB.NET or handle exceptions thrown by F# modules.

- **Automatic Memory Management:** The Garbage Collector (GC) runs non-deterministically in the background, automatically reclaiming memory occupied by objects that are no longer reachable by the application.
- **Simplified Deployment:** Applications can often be deployed simply by copying files to the target directory (XCOPY deployment). Assemblies are self-describing via metadata, eliminating the need for complex Windows Registry configurations.
- **Robust Security:** Features like Code Access Security (CAS) and role-based security ensure that code is only granted permissions verified by administrative policies.

Disadvantages

- **Platform Dependency:** The classical .NET Framework is natively restricted to the Microsoft Windows operating system (though this was addressed in modern iterations like .NET Core / .NET 5+).
- **Resource Overhead:** The abstraction layers, JIT compilation process, and continuous background runtime monitoring consume more system memory and CPU cycles compared to unmanaged native code like pure C++.
- **Reverse Engineering Vulnerability:** Because CIL code retains rich metadata and structural logic, compiled assemblies can be easily decompiled into readable source code using reflection tools, necessitating third-party obfuscation for proprietary logic.

3. The Framework Class Library (FCL)

The Framework Class Library (FCL) is a comprehensive collection of reusable object-oriented types (classes, interfaces, structures, enumerations, and delegates) integrated directly into the .NET platform.

The "Why" Behind the FCL

Without a standard library, developers would have to write low-level code for basic tasks like reading a text file, parsing a date, or opening a network socket. The FCL provides tested, optimized, and secure abstractions for these operations, allowing developers to focus on application-specific business logic rather than foundational plumbing.

3.1 Core Elements of the FCL

Data Types

The .NET Framework enforces a Common Type System (CTS), ensuring that data types are unified across all supported programming languages. For example, an integer in C# (`int`) and an integer in VB.NET (`Integer`) both map to the exact same underlying FCL type: `System.Int32`.

- **Value Types:** Stored directly on the memory stack. They hold their data directly and include primitive types (integers, booleans, floats) and custom structures.
- **Reference Types:** Stored on the memory heap, with a reference (pointer) to that memory location kept on the stack. Classes, interfaces, strings, and arrays are reference types.

Exceptions

Exceptions are the standardized mechanism for handling runtime errors in a structured, object-oriented manner. Instead of relying on return codes, operational failures throw an object deriving from `System.Exception`.

- **Purpose:** Separates error-handling logic from regular business logic and ensures that errors cannot be silently ignored.
- **Mechanism:** Code is wrapped in a try block. If an error occurs, execution halts and jumps to a matching catch block. A finally block guarantees execution of clean-up code (like closing file handles) regardless of whether an error occurred.

Events

Events provide a formal notification system that allows an object to signal other objects when a specific action occurs (e.g., a button click or a completed data download).

- **Purpose:** Enables loosely coupled architectures using the Publisher-Subscriber pattern.
- **Mechanism:** Events are built on top of Delegates (type-safe function pointers). The publisher class raises the event, and any subscriber class that has registered a matching method via the delegate receives the notification and executes its event handler.

Attributes

Attributes are declarative tags used to inject runtime-readable metadata directly into code elements (assemblies, classes, methods, or properties).

- **Purpose:** Allows developers to embed instructions or descriptive data without changing the underlying logic of the code.
- **Real-World Use:** Marking a class as serializable (`[Serializable]`), defining routing paths in web applications, or specifying obsolete methods (`[Obsolete]`). This metadata is read dynamically at runtime using a process called Reflection.

Collections

Collections are specialized classes designed for data storage and retrieval, offering dynamic memory sizing and advanced management algorithms beyond standard arrays.

- **Non-Generic Collections:** Found in `System.Collections` (e.g., `ArrayList`, `HashTable`). They store objects as base `System.Object` types, requiring costly processing overhead known as boxing (converting value types to reference types) and unboxing.
- **Generic Collections:** Found in `System.Collections.Generic` (e.g., `List`, `Dictionary<TKey, TValue>`). They enforce strong type safety at compile-time, eliminating boxing overhead and preventing runtime casting errors.

Remote Calls

Remote calls allow a .NET application to communicate with objects executing in different application domains, processes, or across physical network boundaries.

- **Evolution:** Originally handled by .NET Remoting, this concept evolved into Windows Communication Foundation (WCF).
- **Purpose:** Facilitates distributed computing. Objects can be accessed by reference across a network proxy, making remote method invocations appear as though they are local function calls to the consuming application.

Threading

Threading enables concurrent execution paths within a single application process. The core functionality is managed via the `System.Threading` namespace.

- **Purpose:** Maximizes CPU utilization and maintains application responsiveness. For example, a lengthy database query can run on a background thread while the main thread keeps the user interface responsive.
- **Control:** The FCL provides objects like `Thread`, `ThreadPool`, and synchronization primitives (`Monitor`, `Mutex`, `Semaphore`) to manage concurrency and prevent race conditions where multiple threads access shared data simultaneously.

Web Services

Web Services provide standard mechanisms for applications to exchange data over the internet using open communication protocols.

- **Purpose:** Enables application-to-application integration across disparate platforms (e.g., a .NET server communicating with a Java client).
- **Technologies:** Initially implemented as ASP.NET Web Services (ASMX) using XML and SOAP protocols. It later matured into WCF and modern RESTful ASP.NET Web APIs utilizing lightweight JSON formatting.

4. Working with the .NET FCL: Namespaces

With tens of thousands of individual types residing within the Framework Class Library, organizational chaos and naming collisions would occur without a strict management structure. The solution to this problem is the implementation of Namespaces.

What is a Namespace?

A namespace is a logical naming scheme used to group related types (classes, interfaces, structures) into a hierarchical tree structure. Namespaces do not correspond directly to physical file boundaries or assembly files; rather, they provide organizational boundaries.

The "Why" Behind Namespaces

1. Preventing Scope Collisions: Two distinct classes can share the exact same name provided they reside in different namespaces. For example, a developer can create a custom class named File without conflicting with the native FCL file handler, because the FCL class is fully qualified as System.IO.File.
2. Logical Discovery: Namespaces group functionality by domain, allowing developers to navigate the massive library intuitively.

Importing Namespaces

Writing fully qualified names (e.g., System.Console.WriteLine("Hello")); repetitively clutters source code. Developers use keyword directives at the top of code files to import the namespace into the current scope. Once imported, the child classes can be referenced directly.

- C# Syntax: using System.IO;
- VB.NET Syntax: Imports System.IO

Common FCL Core Namespaces

- System: The root namespace containing fundamental base types (String, Int32, Object, Console, Math).
- System.Collections.Generic: Contains high-performance, strongly-typed generic collection classes.
- System.IO: Contains types that allow reading and writing to files and data streams.
- System.Data: Contains the ADO.NET architecture utilized for querying and managing relational databases.
- System.Net: Provides simple programming interfaces for standard network protocols.

Unit 2: Visual Basic .NET Language Fundamentals and Programming Constructs

1. Core Language Elements

A robust understanding of Visual Basic .NET (VB.NET) begins with its foundational syntax and structural rules. VB.NET is a strongly-typed, object-oriented language running on the .NET Framework, designed to combine logical readability with enterprise-grade power.

1.1 Data Types

Data types specify the size and type of values that can be stored in an identifier (variable). VB.NET categorizes data types natively into Value Types and Reference Types based on how they are allocated in memory.

- Primitive Value Types: Integers (**Integer**, **Short**, **Long**), floating-point numbers (**Single**, **Double**, **Decimal**), characters (**Char**), and booleans (**Boolean**). These are stored directly on the stack.
- Reference Types: Strings (**String**), arrays, and custom classes. These store a memory address on the stack that points to the actual data allocated on the managed heap.
- The "Why" Behind Strong Typing: Enforcing specific data types prevents accidental data corruption (for example, attempting to perform mathematical division on a text string) and allows the runtime to allocate the exact amount of required memory, optimizing performance.

1.2 Operators

Operators are symbols or keywords that trigger specific operations on one or more operands.

- Arithmetic Operators: Perform standard mathematical calculations. Includes addition (+), subtraction (-), multiplication (×), regular division (/ for floating-point results), integer division (\ which drops the remainder), modulus (**Mod** for returning only the remainder), and exponentiation (^).
- Relational Operators: Compare two values and return a boolean result. Includes equal to (=), not equal to (<>), greater than (>), less than (<), greater than or equal to (>=), and less than or equal to (<=).
- Logical Operators: Combine multiple boolean expressions. Standard logical operators include **And**, **Or**, and **Not**. VB.NET also provides short-circuiting logical operators: **AndAlso** and **OrElse**.
- The "Why" Behind Short-Circuiting: If the first condition of an **AndAlso** statement evaluates to false, the runtime skips evaluating the second condition entirely. This prevents errors (like null reference exceptions) and saves processing cycles.
- Assignment Operators: Assign values to variables, often combined with arithmetic operations for brevity (e.g., =, +=, -=).

1.3 Modifiers

Modifiers are keywords used to change the default behavior or visibility of types and type members.

Access Modifiers

Control the visibility of code elements to enforce object-oriented encapsulation.

- **Public:** The code is accessible from any other code in the same assembly or another assembly that references it.
- **Private:** The code is accessible only within the exact same class or structure.
- **Protected:** The code is accessible within the same class, or in a class that inherits from that base class.
- **Friend:** The code is accessible only within files located in the same compiled assembly (equivalent to "internal" in other languages).

Non-Access Modifiers

Control operational behavior and instantiation rules.

- **Shared:** Declares a member that belongs to the class type itself rather than to a specific object instance.
- **ReadOnly:** Indicates that a variable can only be assigned a value during its declaration or within the class constructor.
- **Const:** Declares a compile-time constant value that cannot be modified anywhere in the application lifecycle.

1.4 Directives

Preprocessor directives give instructions to the compiler to process structural information before compilation occurs.

- **#Const:** Defines a conditional compiler constant directly within a code file.
- **#If...#Else...#End If:** Instructs the compiler to include or exclude specific blocks of code based on whether a conditional constant evaluates to true. This is heavily utilized for separating debugging routines from production builds.
- **#Region...#End Region:** Allows developers to define a block of code that can be expanded or collapsed within visual code editors, improving document navigation.

2. Programming Constructs: Decision Making, Case, and Iteration

Programming constructs direct the flow of execution based on dynamic runtime evaluations.

2.1 Decision Making Constructs

Decision structures evaluate one or more conditions to determine the appropriate execution path.

- **If...Then...End If:** Executes a block of code strictly if a specified condition evaluates to True.
- **If...Then...Else...End If:** Provides a secondary path of execution when the primary condition evaluates to False.
- **Elseif:** Allows testing of multiple sequential conditions within a single, unified decision block without requiring deep nesting.

2.2 Case Selection Constructs

When an expression must be compared against numerous potential constant values, the **Select Case** statement provides a highly readable alternative to complex **If...Elseif** ladders.

- **Execution Flow:** The **Select Case** expression is evaluated once at the top of the structure. The runtime compares the result against the values specified in individual **Case** statements. When a match is found, the associated code executes, and the structure automatically exits at **End Select**.
- **Syntax Advantages:** Unlike other languages, VB.NET does not require explicit break statements to prevent execution from falling through to the next case. It also natively supports evaluating ranges (**Case 1 To 5**) and conditional comparisons (**Case Is > 10**). A **Case Else** block handles un-matched scenarios.

2.3 Iteration Constructs (Loops)

Iteration allows a set of instructions to execute repeatedly until a specific termination condition is met. The lifecycle of any loop involves four distinct phases:

1. **Initialization:** Setting the starting value of the loop control variable.
2. **Condition Evaluation:** Testing if the loop should continue executing.
3. **Body Execution:** Running the block of code inside the loop bounds.
4. **Iteration Step:** Modifying the loop control variable to progress toward the termination condition.

Types of Loops

- **For...Next Loop:** Used for definite iteration, where the exact number of cycles is predetermined. The initialization, target bound, and optional step value (**Step**) are defined in the header.

- **While...End While Loop:** A pre-test loop used for indefinite iteration. The condition is evaluated before the body executes; if the condition is initially false, the inner code never executes.
 - **Do While...Loop / Do...Loop While:** Can be structured as either pre-test or post-test loops. Placing the condition at the end (**Loop While**) guarantees the loop body executes at least once.
 - **For Each...Next Loop:** Specifically optimized to iterate through elements in an array or collection object cleanly, abstracting away manual numeric index tracking.
-

3. Arrays

An array is a data structure used to store a fixed-size sequential collection of elements sharing the same data type.

The "Why" Behind Arrays

Without arrays, storing fifty related numeric sensor readings would require declaring fifty distinct variables. Arrays allow developers to group these related data points under a single variable identifier and access them dynamically using zero-based numerical indexing.

Memory Allocation Process

1. **Array Declaration:** Declaring an array variable allocates space on the stack for a reference pointer. In VB.NET, standard sizing defines the upper bound, meaning an array defined as **Dim numbers(4) As Integer** holds five total elements (indices 0 through 4).
2. **Heap Allocation:** The runtime allocates contiguous memory blocks on the managed heap sufficient to hold the specified number of elements based on their underlying data type size.
3. **Reference Mapping:** The stack pointer is updated with the memory address pointing to the starting element on the heap.

Array Classifications

- **Single-Dimensional Arrays:** A simple linear list of elements accessed via a single index enclosed in parentheses.
- **Multidimensional Arrays:** Data stored in a strict grid format with rows and columns. Declared using comma-separated dimensions, such as **Dim grid(2, 2) As Integer**.
- **Jagged Arrays:** An array of arrays. Each individual element of the primary array is itself an array reference, allowing individual child rows to have varying lengths.

Advantages and Disadvantages of Arrays

- **Advantages:** Immediate data retrieval due to direct index access; continuous memory allocation optimizes CPU caching mechanisms.
 - **Disadvantages:** Fixed sizing dictates that standard arrays cannot grow or shrink dynamically after instantiation; inserting elements requires building a new array or utilizing expensive manual shifting logic.
-

4. Procedures and Functions (Methods)

In VB.NET, operational modules of code are split into two distinct sub-types: Procedures (**Sub**) and Functions (**Function**).

4.1 Sub Procedures vs. Functions

- **Sub Procedure:** A block of code that performs an action but does not return a value to the calling code. The definition begins with **Sub** and terminates with **End Sub**.
- **Function:** A block of code designed to perform calculations or processing that returns a specific value back to the caller. The definition requires an **As** clause at the end of the signature to specify the return data type, and utilizes the **Return** keyword to pass the data back before reaching **End Function**.

4.2 Parameter Passing Mechanisms

VB.NET supports two core protocols for passing arguments into methods:

1. **ByVal** (Pass by Value): This is the default mechanism. The runtime creates a copy of the actual argument's value and passes it into the method parameter. Any internal modifications made to the parameter do not impact the original variable in the calling code.
 2. **ByRef** (Pass by Reference): Instead of passing a copy of the data, the runtime passes a pointer to the original variable's exact memory address. Any modifications made to the parameter inside the method directly alter the underlying variable in the calling code.
-

5. Standard Library Functions

The Framework Class Library provides built-in VB.NET utilities to handle common textual manipulation, mathematical processing, and type-casting tasks.

5.1 String Functions

Strings in .NET are immutable; modifying a string creates a new instance in memory.

- **Len(string)** or **string.Length**: Returns the total number of characters in the string.
- **string.Substring(startIndex, length)**: Extracts a targeted sequence of characters from the base string.
- **string.Replace(oldValue, newValue)**: Swaps specified characters or words with new values.
- **string.ToUpper()** / **string.ToLower()**: Standardizes character casing for consistent evaluation.
- **string.Split(delimiter)**: Converts a single string into an array of substrings separated by a chosen character.

5.2 Date and Time Functions

Used to capture and manipulate temporal values.

- **Now**: Retrieves the current system date and time.
- **Today**: Retrieves the current system date with the time component set to midnight.
- **date.AddDays(count)**: Generates a new date value shifted forward or backward by the specified number of days.
- **date.ToString(format)**: Outputs the date object as a formatted string (e.g., "dd/MM/yyyy").

5.3 Math Functions

Accessed via the **System.Math** class for highly optimized mathematical operations.

- **Math.Abs(value)**: Returns the absolute positive value of a number.
- **Math.Pow(base, exponent)**: Raises a number to a specified power.
- **Math.Sqrt(value)**: Calculates the square root of a number.
- **Math.Round(value, decimals)**: Rounds a floating-point value to the nearest specified decimal place.

5.4 Data Conversion Functions

VB.NET provides extensive built-in casting operators to transition data cleanly across the Common Type System.

- **CInt(expression)**: Converts a valid expression into an Integer.
- **CDbl(expression)**: Converts an expression into a Double.
- **CStr(expression)**: Converts an expression into a String representation.
- **CType(expression, type)**: A highly versatile casting operator used to convert an expression into any specified object or primitive type.

- **Integer.TryParse(string, result):** Safely evaluates if a string can be converted to an integer, returning True or False without throwing application-crashing exceptions if the data format is invalid.
-

Unit 3: Object-Oriented Programming Systems (OOPS) in Visual Basic .NET

1. Core Features of OOPS

Object-Oriented Programming (OOP) is a foundational software design paradigm that organizes logic around "objects" rather than raw actions, and data rather than pure logic. In Visual Basic .NET (VB.NET), OOP enables enterprise-scale applications by modeling real-world entities and their interactions.

The "Why" Behind OOP

Historically, procedural programming treated data and functions separately. As software grew past thousands of lines of code, global data was frequently mutated by unexpected functions, resulting in fragile "spaghetti code." OOP solves this by bundling data and related behaviors into secure, autonomous units. The real-world purpose of OOP is to manage architectural complexity, facilitate code reuse, and allow distributed teams to work on modular components independently without breaking global application states.

The Four Pillars of OOP

1. **Encapsulation:** The bundling of data (variables) and methods that operate on that data into a single unit (a class), while restricting direct outside access to internal states.
 2. **Abstraction:** Hiding internal implementation details and exposing only the necessary operational interfaces to the consuming code.
 3. **Inheritance:** A mechanism where a new class derives its structural attributes and behaviors from an existing class, establishing an "is-a" relationship.
 4. **Polymorphism:** The ability of different objects to respond uniquely to the same method call depending on their specific class type.
-

2. Declaring Classes, Variables, and Properties

To utilize OOP in VB.NET, developers must define the templates from which objects are constructed.

2.1 Classes vs. Objects

- **Class:** A static blueprint, template, or custom data type that defines the characteristics and behaviors common to all instances of that type. Declared using the **Class** and **End Class** keywords.
- **Object:** A dynamic, living instance of a class allocated in the computer's memory during runtime.

Step-by-Step Instantiation Process

Creating an object from a class involves three distinct runtime phases:

1. **Declaration:** A variable of the class type is declared on the stack, initially holding a null reference (**Nothing** in VB.NET).
2. **Allocation:** The **New** keyword triggers the Common Language Runtime (CLR) to allocate contiguous memory on the managed heap sufficient to store all internal variables of the class.
3. **Initialization and Linking:** The class constructor (**Sub New**) executes to set initial variable states, and the heap memory address is assigned back to the stack variable.

2.2 Public and Private Variables (Fields)

Variables declared directly inside a class are known as fields. Their accessibility dictates the security of the object's internal state.

- **Private Variables:** Defined using the **Private** modifier. These variables are completely hidden from outside code and can only be read or modified by methods internal to the exact same class. This represents the foundation of encapsulation.
- **Public Variables:** Defined using the **Public** modifier. These variables can be directly read and overwritten by any external code referencing the object.
- **Structural Rule:** Exposing public fields is considered poor architectural practice because it bypasses validation logic, allowing external code to inject invalid states (e.g., setting an account balance field directly to a negative number).

2.3 Properties

Properties act as smart gatekeepers. To the outside world, a property looks and behaves exactly like a standard variable. Internally, however, it is implemented as a pair of specialized access methods known as Getters and Setters.

- **Get Block:** Executes automatically when external code attempts to read the property value. It securely returns the underlying private field data.
- **Set Block:** Executes automatically when external code attempts to assign a value to the property. It captures the incoming value using the implicit **Value** keyword,

allowing developers to execute validation logic before updating the underlying private field.

Advantages and Disadvantages of Properties

- Advantages: Enforces strict data validation; allows read-only properties (omitting the **Set** block) or write-only properties; maintains binary compatibility if internal implementations change later.
 - Disadvantages: Introduces a minor execution overhead compared to direct field access due to method invocation.
-

3. Inheritance

Inheritance allows a new class (the Derived Class or Child Class) to absorb the data structures and behaviors of an existing class (the Base Class or Parent Class). In VB.NET, this is achieved using the **Inherits** keyword placed immediately on the line below the class declaration.

The "Why" Behind Inheritance

Without inheritance, creating classes for a Manager, a Technician, and an Intern would require duplicating shared logic (Name, EmployeeID, HireDate) across all three files. Inheritance allows developers to consolidate shared logic into a single **Employee** base class. Child classes inherit these attributes automatically and only need to define behaviors unique to their specific roles.

Structural Rules of Inheritance in VB.NET

- Single Inheritance: A class in VB.NET can inherit from only one base class directly. Multiple class inheritance is prohibited to prevent architectural ambiguities (such as the "Diamond Problem").
- Transitive Nature: If Class C inherits from Class B, and Class B inherits from Class A, Class C automatically inherits all members of both Class B and Class A.
- Accessing Base Logic: Child classes can access visible base class members using the **MyBase** keyword.

Advantages and Disadvantages of Inheritance

- Advantages: Maximizes code reusability; establishes clear logical hierarchies; facilitates polymorphic design patterns.
- Disadvantages: Creates tight coupling between parent and child classes; changes made to a base class can inadvertently break functional logic in deeply nested derived classes.

4. Polymorphism: Overloading vs. Overriding

Polymorphism allows objects to be treated as instances of their parent class while executing custom behaviors specific to their derived types. This is divided into compile-time and runtime mechanisms.

4.1 Method Overloading (Compile-Time Polymorphism)

Overloading occurs when a single class contains multiple methods sharing the exact same name, but possessing different parameter signatures (different number of parameters, different data types, or a different order of types). In VB.NET, this is explicitly marked using the **Overloads** keyword.

- The "Why": Improves API readability. Instead of creating distinct methods like **PrintInteger**, **PrintString**, and **PrintDouble**, a developer exposes a unified **Print** method, leaving the compiler to map the call to the correct implementation based on the arguments provided.
- Resolution Phase: The compiler determines precisely which method to invoke during compilation by matching the calling signature against the available definitions.

4.2 Method Overriding (Runtime Polymorphism)

Overriding occurs when a child class intercepts a method inherited from a parent class and replaces its logic entirely. In VB.NET, the derived method must match the parent method's signature exactly and utilize the **Overrides** keyword.

- The "Why": Allows a general base class contract to execute highly specialized behaviors. For example, a base class **Shape** might define a **CalculateArea** method. Derived classes like **Circle** and **Rectangle** override this method to apply their specific geometric formulas.
- Resolution Phase: Resolved dynamically at runtime using Late Binding. The Common Language Runtime checks the actual object instance type created in memory and executes the overridden child logic, even if the object is stored inside a parent-typed variable reference.

5. Virtual Methods and Abstract Classes

To control how inheritance and overriding function across an architecture, base classes apply specific access contracts to their structural templates.

5.1 Virtual Methods (Overridable)

By default, standard methods in a VB.NET base class cannot be overridden by child classes. To permit overriding, the base class method must be explicitly marked with the **Overridable** keyword (equivalent to "virtual" in other languages).

- Behavior: An **Overridable** method provides a fully functional default implementation in the base class. Derived classes have the option to accept this default behavior as-is, or use the **Overrides** keyword to provide customized logic.

5.2 Abstract Classes (MustInherit)

An abstract class is an incomplete structural blueprint that cannot be instantiated directly into an object. In VB.NET, classes are made abstract using the **MustInherit** modifier.

- The "Why": Used to define a rigid architectural template. It guarantees that certain concepts exist conceptually but mandates that complete implementations are left entirely to child classes. For example, a **Vehicle** class is conceptually valid, but you cannot build a generic "vehicle" without knowing if it is a Car, Boat, or Plane.

5.3 Abstract Methods (MustOverride)

Methods inside an abstract class can be marked with the **MustOverride** keyword.

- Behavior: A **MustOverride** method contains no code body (no internal statements and no **End Sub** or **End Function** tags). It simply defines the method signature.
- Architectural Enforcement: Any concrete child class that inherits from the abstract base class is strictly forced by the compiler to override and implement every **MustOverride** method. Failure to do so prevents the application from compiling.

Unit 4: Windows Forms and Graphical User Interfaces (GUI) in VB.NET

1. Introduction to Class Libraries for Windows Forms

In the .NET Framework, developing a desktop application with a Graphical User Interface (GUI) relies heavily on standard class libraries. Instead of interacting directly with the low-level operating system graphics rendering engine (the Win32 API), developers utilize pre-built, object-oriented wrapper classes.

The "Why" Behind GUI Class Libraries

Building a simple application window from scratch using raw operating system code requires hundreds of lines of complex logic just to register a window class, allocate drawing buffers, and track screen coordinates. The Windows Forms Class Library abstracts these complexities. Its primary real-world purpose is to provide rapid application development (RAD) tools, allowing developers to instantiate fully functional windows, dialogs, and controls with minimal code.

Core Namespaces

- **System.Windows.Forms:** Contains the classes for creating desktop applications that take full advantage of the rich user interface features available in the Windows operating system. Every window, button, drop-down list, and dialog box derives from classes within this namespace.
 - **System.Drawing:** Provides access to GDI+ (Graphics Device Interface) basic graphics functionality. It is used to handle rendering logic, such as defining colors, managing fonts, drawing custom geometric shapes, and manipulating images.
-

2. Events and Event Handlers

Windows Forms desktop applications operate on an Event-Driven Programming paradigm. Unlike traditional procedural console applications that execute sequentially from top to bottom, a GUI application initializes its interface and then enters an idle state, waiting for specific external stimuli.

2.1 The Concept of Events

An event is a message sent by an object (such as a button or the form itself) signaling that a specific action has occurred. These actions can be triggered by user interactions (mouse clicks, keystrokes), system notifications (timer ticks), or programmatic state changes.

2.2 Event Handlers

An event handler is a specialized sub procedure designed to intercept an event and execute the appropriate response logic. In VB.NET, event wiring is handled cleanly using declarative keywords.

- **The Handles Keyword:** Placed at the end of a sub procedure's signature to link the method directly to a specific object's event.
- **The WithEvents Keyword:** Used when declaring an object variable at the class level to notify the compiler that the object will raise events that need to be tracked.

Step-by-Step Event Lifecycle

1. **Listener Registration:** During form initialization, the application registers event handler methods with their corresponding control events using underlying delegate mechanisms.
2. **Triggering Action:** The user performs a physical action, such as clicking a graphical button.
3. **OS Interception:** The Windows Operating System detects the hardware interrupt (mouse click) and passes a message to the application's message loop.

4. Event Invocation: The WinForms runtime identifies the target control and raises its default event (e.g., Click).
 5. Handler Execution: The CLR routes execution directly to the linked VB.NET Sub procedure, passing standard event arguments (sender object and event data).
-

3. WinForms and Graphical User Interface (GUI) Design

A Graphical User Interface replaces typing command-line instructions with visual paradigms, utilizing windows, icons, menus, and pointers (WIMP).

The Anatomy of a WinForm

A Windows Form is fundamentally a custom class that inherits from `System.Windows.Forms.Form`. When a developer designs a screen visually in Visual Studio, the IDE automatically generates standard VB.NET code behind the scenes to construct the interface elements.

Lifecycle of a WinForms Application

1. Application Startup: The application's entry point invokes `Application.Run(New Form1())`, establishing a continuous message loop to monitor OS operations.
2. Form Construction: The class constructor (Sub New) executes, immediately calling `InitializeComponent()` to instantiate all placed controls and configure their visual layouts.
3. Form Load Event: The `Form.Load` event fires before the window becomes visible, serving as the standard location to populate data from databases or initialize default variables.
4. Active Display: The window renders on screen, ready for user interaction.
5. Form Closing/Closed: Triggered when the user attempts to exit. Developers can intercept the `FormClosing` event to prompt the user to save unsaved work, optionally canceling the closure.

Advantages and Disadvantages of WinForms

- Advantages: Extremely fast design-time prototyping via drag-and-drop interfaces; highly mature, stable, and well-documented API; highly performant for local, line-of-business desktop tools.
- Disadvantages: Interface scaling can become problematic on modern high-DPI monitors; tight coupling between UI layout logic and back-end code makes rigorous unit testing difficult compared to modern frameworks like WPF (Windows Presentation Foundation).

4. Concept of Controls

In WinForms, every visual interactive element placed onto a form is known as a Control. All controls inherit baseline behaviors, rendering logic, and coordinate management from the base `System.Windows.Forms.Control` class.

4.1 Universal Base Properties

Because controls share a unified base class, they inherit standard configurable properties:

- **Name:** The unique identifier used to reference the control programmatically within code.
- **Text:** The literal string displayed visually inside or alongside the control.
- **Enabled:** A boolean property determining whether the control can respond to user interaction.
- **Visible:** A boolean property determining whether the control is rendered on screen.
- **Size and Location:** Structures defining the width/height and the X/Y Cartesian coordinates relative to the parent container's upper-left corner.

4.2 Universal Base Methods and Events

- **Methods:** `Focus()` forces input focus to the targeted control; `Hide()` and `Show()` dynamically alter visibility states; `Refresh()` forces an immediate visual redraw.
- **Events:** `Click` fires upon activation; `MouseEnter` and `MouseLeave` track cursor proximity; `KeyPress` captures keyboard inputs.

5. Different GUI Controls: Properties, Methods, and Events

To build fully functional interfaces, developers assemble diverse sets of specialized controls.

5.1 Button Control

Used to initiate a distinct command or action when activated.

- **Key Properties:** `FlatStyle` (alters visual rendering themes); `Image` (embeds an icon directly onto the button face).
- **Key Methods:** `PerformClick()` programmatically simulates a physical user click.
- **Primary Event:** `Click`.

5.2 TextBox Control

Provides an input area for users to enter, view, or edit unformatted text strings.

- Key Properties: Multiline (allows input across multiple rows); ReadOnly (prevents editing while allowing text selection); PasswordChar (masks typed input with a specific character for security).
- Key Methods: Clear() empties the contents; AppendText(string) adds text to the end without overwriting existing data.
- Primary Event: TextChanged (fires instantly upon every individual keystroke or programmatic modification).

5.3 Label Control

Displays informative text or instructions that the user cannot directly edit.

- Key Properties: AutoSize (forces the control boundaries to automatically wrap around text length); BorderStyle (adds 3D or flat bounding lines).
- Primary Event: Rarely utilizes events, acting primarily as static interface signposting.

5.4 CheckBox and RadioButton Controls

Used to capture binary selections or choices from a user.

- CheckBox: Allows multi-selection. Multiple checkboxes operate independently of one another.
- RadioButton: Enforces mutually exclusive selection. Placed within the same container, selecting one RadioButton automatically unchecks all others.
- Key Properties: Checked (a boolean flag indicating active state).
- Primary Event: CheckedChanged (fires whenever the selection state toggles).

5.5 ListBox and ComboBox Controls

Displays lists of selectable items to streamline input and prevent typing errors.

- ListBox: Displays an open container showing multiple rows simultaneously. Supports selecting multiple items holding down Ctrl or Shift.
- ComboBox: Consolidates space by displaying a single drop-down line that expands upon activation.
- Key Properties: Items (the internal collection of data entries); SelectedIndex (an integer tracking the zero-based position of the chosen item); SelectedItem (retrieves the actual selected object).
- Key Methods: Items.Add(object) appends new choices dynamically; Items.RemoveAt(index) strips an item from the collection.

- Primary Event: SelectedIndexChanged.

6. Program Logic Using Controls

To illustrate how these controls communicate programmatically within a VB.NET class file, the following plain-text workflow demonstrates the logic for a simplified processing screen containing a TextBox (txtInput), a Button (btnProcess), and a Label (lblOutput).

```
Class Form1 Inherits Form
' Executed automatically when the user clicks the processing button
Private Sub btnProcess_Click(sender As Object, e As EventArgs) Handles btnProcess.Click

    ' Phase 1: Capture and validate inputs
    Dim rawInput As String = txtInput.Text.Trim()

    If rawInput = "" Then
        MessageBox.Show("Please enter a valid value before processing.", "Input Error")
        txtInput.Focus()
        Exit Sub
    End If

    ' Phase 2: Execute domain logic
    Dim characterCount As Integer = rawInput.Length
    Dim formattedResult As String = "Processed successfully. Total characters: " &
characterCount

    ' Phase 3: Update interface state
    lblOutput.Text = formattedResult
    txtInput.Clear()

End Sub
End Class
```

Unit 5: Error Handling and Validation in Windows Forms (VB.NET)

1. Introduction to Errors, Exceptions, and Validations

In software engineering, constructing a robust Graphical User Interface (GUI) requires anticipating that users will enter invalid data, external systems will fail, and internal logic

boundaries will be tested. Unit 5 explores the structural defense mechanisms available in Visual Basic .NET (VB.NET) desktop applications: Errors, Exceptions, and Validations.

The "Why" Behind Defensive Programming

If a desktop application assumes ideal operating conditions, a single missing file or misplaced alphabetical character in a numeric calculation will cause the runtime engine to terminate the process immediately. This sudden crash results in lost data, user frustration, and potential security vulnerabilities.

- The Real-World Purpose: Defensive programming guarantees that an application degrades gracefully. By implementing rigorous input validation and structured exception handling, the application intercepts anomalies, alerts the user constructively, and maintains internal system stability.

2. Types of Errors

Errors represent flaws, defects, or misconfigurations within the software lifecycle. In VB.NET development, errors are broadly categorized into three distinct phases based on when they are detected.

2.1 Syntax Errors (Compile-Time Errors)

Syntax errors occur when source code violates the grammatical rules of the VB.NET compiler.

- Identification Phase: Detected immediately inside the Visual Studio IDE during code drafting or when triggering a build.
- Common Causes: Misspelled keywords (e.g., writing **Dimx** instead of **Dim x**), missing closing tags (omitting **End If**), or calling methods with incorrect argument counts.
- Result: The application is strictly blocked from compiling into an executable assembly until every syntax error is resolved.

2.2 Runtime Errors

Runtime errors occur after the application has successfully compiled and is actively executing in memory.

- Identification Phase: Triggered dynamically when the program attempts an operation that the underlying operating system or hardware cannot execute.
- Common Causes: Attempting to open a file that has been deleted, dividing an integer by zero, or trying to establish a database connection over a disconnected network.
- Result: If unhandled, runtime errors halt execution and generate system-level crash dialogs.

2.3 Logic Errors (Semantic Errors)

Logic errors are the most insidious type of flaw because the code compiles perfectly and executes without crashing, but produces incorrect outcomes.

- **Identification Phase:** Discovered during dedicated software testing, peer reviews, or user reporting.
 - **Common Causes:** Utilizing an incorrect mathematical formula (e.g., using addition instead of multiplication), referencing the wrong variable inside a loop, or setting an incorrect boundary condition (< instead of <=).
 - **Result:** Flawed business outputs, corrupted database entries, or infinite execution loops.
-

3. Types of Exceptions

An Exception is an object-oriented representation of a runtime error. When an operational failure occurs, the Common Language Runtime (CLR) packages the error details (stack trace, error message, origin target) into an object deriving from the base **System.Exception** class.

3.1 System Exceptions (System.SystemException)

These are predefined exceptions thrown directly by the CLR when core execution boundaries are violated.

- **System.NullReferenceException:** Occurs when code attempts to invoke a method or read a property on an object variable that currently points to **Nothing** (null).
- **System.FormatException:** Thrown when the format of an argument does not meet the parameter specifications of the invoked method, such as passing the string "ABC" into **Integer.Parse()**.
- **System.IndexOutOfRangeException:** Triggered when attempting to access an array element using an index that is less than zero or greater than the array's defined upper bound.
- **System.IO.FileNotFoundException:** Occurs when file manipulation classes attempt to read a path that does not exist on the target disk.
- **System.Data.SqlClient.SqlException:** Encapsulates warnings and errors returned directly by a localized or remote SQL Server database.

3.2 Application Exceptions (System.ApplicationException)

These are custom exceptions defined by the software developer to map domain-specific business logic failures.

- The "Why": Standard system exceptions describe technical failures. Custom application exceptions describe business rules violations (e.g., **InsufficientFundsException** or **InvalidInventoryCountException**).
 - Creation: Built by creating a custom class that inherits from the base **Exception** class, allowing developers to embed specialized business logic properties directly into the error payload.
-

4. Structured Exception Handling Mechanism

VB.NET abandons historical, unstructured error trapping (such as **On Error GoTo**) in favor of robust Structured Exception Handling (SEH) utilizing distinct code blocks.

Anatomy of a Try-Catch Block

Structured exception handling isolates risky code from error recovery logic using four fundamental keywords:

1. **Try** Block: Encloses the specific statements that might trigger a runtime exception. The runtime monitors execution closely within these bounds.
2. **Catch** Block: Acts as a targeted filter. If an exception is thrown inside the Try block, normal execution halts, and the runtime scans downward for a Catch block whose parameter matches the specific exception type. Developers can chain multiple Catch blocks from most specific to least specific.
3. **Finally** Block: An optional but highly critical section. Code placed inside the Finally block is guaranteed to execute regardless of whether the Try block succeeded perfectly or crashed into a Catch block. It is the mandatory location for releasing external system resources (closing file streams, terminating database connections).
4. **End Try**: Terminates the structured exception handling scope.

The Throw Keyword

Developers use the **Throw** keyword to manually raise an exception object when a functional condition fails. This can be used to re-throw an intercepted exception up the call stack to a higher-level management component.

5. Introduction to Validations in Windows Forms

While exception handling acts as a safety net for unexpected crashes, Validation serves as the primary line of defense. Validation is the proactive process of inspecting user input to ensure it meets strict formatting, length, and range requirements before any domain calculations or database updates occur.

Prevention vs. Cure

Relying on exception handling to catch bad user input introduces heavy processing overhead because constructing exception objects and unwinding stack frames consumes significant system memory.

- Golden Rule of WinForms: Exceptions should be reserved for exceptional, unavoidable runtime states. User input errors are completely foreseeable and must be intercepted via UI Validation.

Advantages and Disadvantages of Client-Side UI Validation

- Advantages: Immediate visual feedback improves the user experience; protects backend databases from injection attacks and data corruption; saves network bandwidth by stopping bad requests locally.
 - Disadvantages: Must be duplicated on backend server layers in distributed systems, as desktop client validation can theoretically be bypassed by malicious actors interception tools.
-

6. Types of Validations

Windows Forms provides multiple granular methodologies for validating user inputs depending on interface complexity.

6.1 Field-Level Validation (Control Events)

Field-level validation assesses the data inside a specific control the moment the user attempts to interact with it or navigate away.

The Validating and Validated Events

Every WinForms input control inherits two specialized focus-shifting events:

- **Validating** Event: Fires when the control attempts to lose focus. The event handler receives a specialized argument (**CancelEventArgs**). If the input data fails inspection, the developer sets **e.Cancel = True**. This action traps the user inside the control, preventing input focus from shifting to the next field until the error is corrected.
- **Validated** Event: Fires immediately after the **Validating** event completes successfully, signaling that the data is confirmed and safe for local storage.

6.2 Form-Level Validation

Form-level validation defers data evaluation until the user attempts a definitive submission action, such as clicking a "Save" or "Process" button.

- Execution: The submission button's click event handler executes sequential checks across all required controls on the form. If any check fails, processing halts, and user alerts are generated.
- Best Use Case: Ideal for complex workflows where fields depend on one another (e.g., verifying that an entered "End Date" occurs after a "Start Date").

6.3 The ErrorProvider Component

Historically, UI validation failures triggered standard message boxes (**MessageBox.Show**). This interrupted workflow by forcing users to click "OK" repeatedly. The **ErrorProvider** component offers a modern, non-intrusive alternative.

- Mechanism: Dropped onto the form design tray as a non-visual component.
- Behavior: When validation fails, the developer invokes **ErrorProvider.SetError(control, message)**. This renders a small, blinking red warning icon directly adjacent to the offending input control. Hovering the mouse pointer over the icon displays a tooltip containing the exact error correction instructions.

Unit 6: SDI & MDI Applications and File Handling

1. Introduction to SDI and MDI Applications

In desktop software architecture, managing how users interact with multiple documents or views within a Graphical User Interface (GUI) is a critical design decision. Windows Forms in VB.NET supports two primary interface paradigms: Single Document Interface (SDI) and Multiple Document Interface (MDI).

1.1 Single Document Interface (SDI)

An SDI application is structured so that every individual document or workspace opens inside its own entirely independent top-level window.

- The "Why" Behind SDI: As monitors grew larger and multi-monitor setups became standard, users needed the ability to view distinct documents side-by-side across physical screens. SDI delegates window management directly to the host Operating System's taskbar.
- Real-World Examples: Microsoft Notepad, modern web browsers (where each window operates independently), and standard photo viewing utilities.

1.2 Multiple Document Interface (MDI)

An MDI application utilizes a single primary Parent Window (the container) that houses and encapsulates multiple Child Windows. Child windows cannot be dragged outside the physical boundaries of the parent window.

- The "Why" Behind MDI: In complex, document-heavy workflows, opening dozens of separate taskbar windows clutters the operating system interface. MDI consolidates related tasks into a unified workspace, sharing a single main menu bar and toolbar across all active child documents.
- Real-World Examples: Traditional enterprise resource planning (ERP) software interfaces, database management tools (like SQL Server Management Studio), and classic graphics editors.

Step-by-Step Implementation of MDI in VB.NET

Creating an MDI structure requires establishing a strict parent-child relationship between forms:

1. Establish the Container: Set the `IsMdiContainer` property of the main base form (e.g., `frmMain`) to `True`. The visual surface of the form instantly updates to a sunken dark gray workspace.
2. Instantiate the Child Form: In code, create a dynamic instance of the secondary form class (e.g., `Dim newChild As New frmDocument()`).
3. Link Parent to Child: Explicitly assign the current main form instance as the parent by setting the child's `MdiParent` property (`newChild.MdiParent = Me`).
4. Display the Child: Invoke the child form's display method (`newChild.Show()`). It will now render strictly confined within the parent's interior client area.

2. Characteristics and Drawbacks

Choosing between SDI and MDI involves balancing workspace organization against navigation complexity.

2.1 SDI Characteristics and Trade-offs

- Characteristics: Each window maintains its own dedicated menu bar, ribbon, and status bar. Windows are treated as equal peers by the Windows Taskbar.

Advantages

- Highly intuitive for end-users; leverages familiar operating system navigation controls (`Alt+Tab`).
- Fully supports multi-monitor arrangements, allowing unrestricted drag-and-drop positioning.
- Superior fault isolation: if one document window encounters a fatal runtime crash, other open SDI instances often remain unaffected.

Disadvantages

- Consumes significant operating system desktop space and taskbar real estate when numerous files are active.
- Programmatic communication between completely independent, unlinked SDI forms requires complex reference passing architectures.

2.2 MDI Characteristics and Trade-offs

- Characteristics: Centralized application control. The active child window dynamically merges its menus with the parent container's menu bar.

Advantages

- Excellent structural organization for document-centric tasks, keeping workspaces neatly contained.
- Rapid context switching using internal window management arrangements (Cascade, Tile Horizontal, Tile Vertical).
- Efficient resource sharing; common background services and toolbars are loaded only once by the parent.

Disadvantages

- Strict boundary confinement prevents users from dragging child windows to secondary monitors.
- Nested scrollbars can emerge if child windows exceed the parent client boundaries, causing severe user interface clutter.
- Modern interface design guidelines have largely deprecated classical MDI layouts in favor of Tabbed Document Interfaces (TDI), similar to modern web browser tabs.

3. Introduction to File Handling

Desktop applications process data inside the computer's Random Access Memory (RAM). Because RAM is volatile, any variable states, records, or user inputs are permanently destroyed the moment the application closes or power is lost. File Handling is the programmatic mechanism used to achieve Data Persistence by storing data on non-volatile secondary storage media (hard drives, solid-state drives).

3.1 The System.IO Namespace

In VB.NET, all file, directory, and data stream manipulation relies on the System.IO namespace. It provides robust classes for reading and writing raw bytes or formatted character strings.

- Core Classes: File (static helper methods for basic file creation and deletion), Directory (folder management), FileInfo and DirectoryInfo (instance-based metadata tracking).

3.2 The Concept of Streams

The .NET Framework abstracts physical disk operations using Streams. A stream is a linear, sequential flow of data bytes traveling between a source and a destination.

- FileStream: Establishes the raw byte-level pipeline between the application and the physical file on the storage drive.
- StreamReader: A specialized high-level wrapper class that consumes raw bytes from a stream and decodes them into readable human text strings.
- StreamWriter: A wrapper class that translates string characters into raw byte sequences and pushes them down the stream to be written to disk.

Step-by-Step Lifecycle of a File Operation

Safe file handling mandates executing operations in a strict, sequential order to prevent locking errors and data corruption:

1. Path Resolution: Define the precise file system directory path using absolute strings or dynamic system variables.
2. Stream Initialization: Instantiate a FileStream or StreamWriter object, explicitly defining the target access mode (Create, Append, Open).
3. Execution: Transmit data using sequential ReadLine or WriteLine methods.
4. Buffer Flushing: Force any cached data remaining in memory buffers to write to the physical disk immediately using the Flush() method.
5. Resource Disposal: Invoke the Close() or Dispose() method. This releases the operating system's exclusive file lock, allowing other external applications to access the document safely.

4. File Handling Program Implementation

The following complete VB.NET logic demonstrates how to construct a practical file handling application. This class captures text input from a user, appends it safely to a persistent log file on disk, and subsequently reads the entire document back into the interface.

```
Imports System.IO
Class Form1 Inherits Form
' Define a constant path for the data file
```

```

Private Const FILE_PATH As String = "C:\ApplicationLogs\UserNotes.txt"

' Programmatic logic to Save Data to Disk
Private Sub btnSave_Click(sender As Object, e As EventArgs) Handles btnSave.Click

    Dim userInput As String = txtInput.Text.Trim()

    If userInput = "" Then
        MessageBox.Show("Cannot save empty records.", "Validation Error")
        Exit Sub
    End If

    ' Phase 1: Ensure directory path exists safely
    Dim folderPath As String = Path.GetDirectoryName(FILE_PATH)
    If Not Directory.Exists(folderPath) Then
        Directory.CreateDirectory(folderPath)
    End If

    ' Phase 2: Initialize StreamWriter within a secure Try-Catch block
    Dim writer As StreamWriter = Nothing

    Try
        ' Passing True as the second parameter enables Append Mode
        writer = New StreamWriter(FILE_PATH, True)

        ' Write timestamped entry to the file stream
        writer.WriteLine(DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss") & " - " & userInput)
        writer.Flush()

        MessageBox.Show("Record appended successfully.", "Success")
        txtInput.Clear()

    Catch ex As IOException
        MessageBox.Show("File access error: " & ex.Message, "IO Error")
    Finally
        ' Phase 3: Guaranteed resource cleanup
        If writer IsNot Nothing Then
            writer.Close()
        End If
    End Try

End Sub

' Programmatic logic to Read Data from Disk

```

```
Private Sub btnLoad_Click(sender As Object, e As EventArgs) Handles btnLoad.Click
```

```
    If Not File.Exists(FILE_PATH) Then
```

```
        MessageBox.Show("No saved records found on disk.", "File Missing")
```

```
        Exit Sub
```

```
    End If
```

```
    Dim reader As StreamReader = Nothing
```

```
    Try
```

```
        ' Initialize StreamReader to pull data from file
```

```
        reader = New StreamReader(FILE_PATH)
```

```
        ' Read all contents from stream to EOF (End of File)
```

```
        Dim fileContents As String = reader.ReadToEnd()
```

```
        ' Display data inside multi-line interface element
```

```
        txtDisplay.Text = fileContents
```

```
    Catch ex As Exception
```

```
        MessageBox.Show("Failed to read file: " & ex.Message, "Read Error")
```

```
    Finally
```

```
        If reader IsNot Nothing Then
```

```
            reader.Close()
```

```
        End If
```

```
    End Try
```

```
End Sub
```

```
End Class
```

Unit 8: Component Object Model (COM), Custom Controls, and Web Programming

1. Introduction to the Component Object Model (COM)

The Component Object Model (COM) is a foundational, language-neutral binary interface standard developed by Microsoft. Prior to the introduction of the .NET Framework, COM served as the primary architecture for building integrated, reusable software components across the Windows operating system.

The "Why" Behind COM

In early software engineering, applications were monolithic blocks of code. Sharing functionality between applications written in different programming languages (such as C++ and Visual Basic 6.0) was exceptionally difficult. COM was created to enable cross-language component reuse and inter-process communication. Its real-world purpose is to allow

disparate software modules to connect and interact at the binary level. For example, COM is the underlying technology that allows a user to directly embed an interactive Microsoft Excel spreadsheet inside a Microsoft Word document.

Core Architectural Principles

- **Interfaces as Contracts:** All communication with a COM component occurs exclusively through standardized interfaces. The foundational interface implemented by every COM object is known as IUnknown. It manages object lifetime and provides interface discovery mechanisms.
- **Globally Unique Identifiers (GUIDs):** To prevent naming collisions across the global software ecosystem, COM components and their specific interfaces are assigned unique 128-bit integer identifiers known as Class IDs (CLSIDs) and Interface IDs (IIDs).
- **Reference Counting:** Because COM objects operate outside managed runtimes with automatic garbage collection, they rely on strict manual reference counting. When an application connects to a COM object, an internal counter increments. When the application disconnects, the counter decrements. Once the reference count reaches zero, the object automatically destroys itself to free system memory.

Interoperability with the .NET Framework

Modern VB.NET applications execute inside the Common Language Runtime (CLR), whereas classic COM components operate in an unmanaged native memory space. Communication between these two environments requires bridging layers known as COM Interop.

1. **Runtime Callable Wrapper (RCW):** Allows a VB.NET managed application to consume an unmanaged COM component. The CLR generates a proxy class that makes the classic COM component appear and behave exactly like a native .NET object.
2. **COM Callable Wrapper (CCW):** Allows legacy unmanaged COM applications to consume modern .NET assemblies. The wrapper exposes .NET classes as standard COM interfaces, handling data marshalling transparently across the boundary.

2. Fundamentals of Custom Controls and User Controls

When building desktop applications in VB.NET, developers are not restricted to the standard library of native tools. The framework allows for the creation of customized graphical components tailored to exact enterprise requirements. These are categorized primarily into Custom Controls and User Controls.

2.1 User Controls (Composite Controls)

A User Control is a graphical component created by assembling multiple existing standard controls (like buttons, labels, and textboxes) onto a reusable design surface. User Controls inherit directly from the `System.Windows.Forms.UserControl` base class.

- The "Why" Behind User Controls: Consider an enterprise application that requires a standardized "Customer Address Block" (containing inputs for Street, City, State, and Postal Code) across ten different application screens. Duplicating these controls and their validation logic ten times creates code bloat and maintenance hazards. Building a unified User Control allows developers to encapsulate this layout and reuse it simply by dragging a single icon onto any form.

Step-by-Step User Control Creation Lifecycle

1. **Class Definition:** Add a new User Control file to the VB.NET project. The IDE generates a blank container surface similar to a standard form.
2. **Interface Layout:** Drag and drop standard child controls onto the surface and configure their visual anchoring and docking properties.
3. **Encapsulation:** Set the internal child controls' access modifiers to `Private`. External forms should never directly read or manipulate child textboxes or labels.
4. **Compilation:** Build the application solution. The newly created User Control automatically populates inside the Visual Studio Toolbox.
5. **Consumption:** Drag the compiled User Control from the Toolbox onto standard application forms, where it behaves as a single unified entity.

2.2 Extended Custom Controls

Unlike composite User Controls, an Extended Custom Control modifies the fundamental drawing logic or core behavior of a single existing control class. This is achieved by inheriting directly from a specific control class, such as inheriting from `System.Windows.Forms.TextBox`.

- Real-World Use Case: Creating a `NumericTextBox` class that automatically intercepts keyboard input events, overriding standard logic to strictly block any non-numeric character strokes before they render on screen.

Advantages and Disadvantages of Custom/User Controls

- **Advantages:** Enforces rigorous code reuse; maintains uniform graphical designs across large applications; encapsulates validation rules within secure, isolated components.
 - **Disadvantages:** Increases initial development overhead; custom logic bugs within shared controls propagate instantly across every referencing application window.
-

3. Events and Properties in User Controls

To make a User Control genuinely flexible, it must be able to accept configuration instructions from its host container (Parent Form) and broadcast notifications back out when internal interactions occur.

3.1 Exposing Custom Properties

Because internal controls are hidden behind private scope boundaries, developers use public Property blocks to act as secure bridges for configuration.

- **Implementation Logic:** If a User Control contains a header label, external forms cannot access the label directly. Instead, the User Control exposes a public property called `HeaderText`. When the external form writes a string to `HeaderText`, the internal `Set` block intercepts the string and updates the private label's text safely.

3.2 Defining and Raising Custom Events

Standard internal control events (like clicking a sub-button inside the User Control) are trapped inside the component. To alert the parent form that an important operation has occurred, the User Control must implement custom events.

Step-by-Step Custom Event Wiring

1. **Event Declaration:** At the class level of the User Control, define the public interface signature using the `Event` keyword (e.g., `Public Event AddressValidated(sender As Object, isValid As Boolean)`).
2. **Internal Interception:** Write a private event handler inside the User Control to catch the internal child control's physical event (such as a validation button click).
3. **Event Broadcasting:** Within that internal handler, use the `RaiseEvent` keyword to broadcast the message out to the host application (e.g., `RaiseEvent AddressValidated(Me, True)`).
4. **External Handling:** In the host Parent Form, instantiate the User Control using the `WithEvents` keyword, or wire it dynamically using the `Handles` keyword to execute logic when the broadcast is received.

4. Introduction to Web Programming in .NET

While Windows Forms handles client-heavy desktop execution, enterprise environments frequently require distributed accessibility. The .NET Framework provides a dedicated web application architecture known as ASP.NET.

The "Why" Behind Web Programming

Desktop software requires physical installation on every individual user machine, creating substantial logistical burdens during software updates. Web applications execute centrally on a secure web server. The real-world purpose of web programming is to achieve platform independence and instant deployment updates; clients access the application globally using any standard web browser without installing local executable files.

4.1 Client-Server Architecture and the Request-Response Lifecycle

Web programming operates over the internet using the Hypertext Transfer Protocol (HTTP). Unlike persistent desktop applications, web applications follow a disconnected, stateless lifecycle.

1. **User Request:** The client browser initiates communication by transmitting an HTTP Request across the network to the targeted web server.
2. **Server Processing:** The IIS (Internet Information Services) web server receives the request and routes it to the ASP.NET runtime engine. The application compiles database queries, executes business logic, and dynamically generates raw HTML, CSS, and JavaScript.
3. **Response Delivery:** The server packages this markup into an HTTP Response and transmits it back to the client browser.
4. **Stateless Disconnection:** Immediately after delivering the response, the server severs the connection and drops all knowledge of the client from its active memory.

4.2 Overcoming HTTP Statelessness: State Management

Because the server treats every page request as an entirely new, isolated event, web applications must utilize specific State Management mechanisms to remember user data (such as login credentials or shopping cart items) across navigation clicks.

- **Client-Side State Management:** Stores data directly inside the user's browser. Includes Cookies (small text files saved to the local hard drive) and ViewState (hidden encrypted form fields embedded directly into the HTML markup to retain control values during page reloads).
 - **Server-Side State Management:** Stores data securely within the web server's memory. Includes Session State (dedicated memory spaces assigned to a single unique user visitor) and Application State (global memory spaces shared simultaneously among all active users of the website).
-

Unit 1: Summary and Core Comparisons

The following table summarizes the key components of the .NET Framework Technology discussed in this unit, serving as a quick reference guide for review.

Concept / Component	Primary Responsibility	Real-World Advantage	Practical Limitation
Common Language Runtime (CLR)	Manages execution, JIT compilation, memory, and security.	Eliminates manual memory management; ensures runtime safety.	Consumes baseline system resources and processing overhead.
Common Type System (CTS)	Standardizes data type representations across all languages.	Seamless language interoperability and cross-language inheritance.	Strict typing rules require explicit conversions in mixed environments.
Value Types vs. Reference Types	Stack allocation (Value) vs. Heap allocation with Stack Pointer (Reference).	Highly optimized memory access tailored to data lifecycle needs.	Mismanagement leads to excessive Garbage Collection pressure.
Generic Collections	Type-safe dynamic data storage (e.g., Lists, Dictionaries).	Eliminates runtime casting errors and expensive boxing operations.	Requires initial architectural awareness of exact data schemas.
Namespaces	Hierarchical logical grouping of classes and interfaces.	Prevents naming collisions and organizes code discoverability.	Deeply nested hierarchies can require extensive using directives.

Unit 2: Summary and Core Comparisons

The following table contrasts the core operational structures and methodologies distinct to Visual Basic .NET discussed throughout this unit.

Construct / Feature	Primary Characteristic	Optimal Use Case	Structural Trade-off
Sub vs. Function	Sub performs actions; Function returns calculated data.	Use Sub for UI event handlers; use Function for reusable math or data queries.	Functions require strict return paths to prevent unassigned default values.
And vs. AndAlso	Standard Bitwise/Logical vs. Short-Circuiting Logical.	Default to AndAlso for standard conditional logic to maximize performance.	And forces evaluation of all conditions, risking runtime exceptions on null checks.
Select Case vs. If-Else	Evaluates a single target against distinct matches or ranges.	Use Select Case for evaluating complex status codes or numeric boundaries.	Cannot easily evaluate independent, completely unrelated boolean expressions.
ByVal vs. ByRef	Passes data copies vs. passes actual memory references.	Default to ByVal to protect global application state from unintended side effects.	ByRef variables can mutate source variables unexpectedly during debugging.
VB Array Upper Bounds	Dimensions define the maximum accessible index limit.	Direct mapping to mathematical bounds (e.g., index 4 represents the 5th element).	Can cause logic confusion for developers accustomed to length-based allocations.

Unit 3: Summary and Core Comparisons

The following table consolidates the critical Object-Oriented Programming mechanisms, keywords, and structural behaviors discussed throughout this unit.

Concept / Feature	VB.NET Keyword	Core Lifecycle Behavior	Primary Architectural Purpose
Class vs. Object	Class / New	Static Code Blueprint vs. Dynamic Heap Allocation.	Separates design-time definition from runtime state management.

Field vs. Property	Private / Property	Raw data storage vs. Encapsulated Get/Set logic blocks.	Protects internal object states from external corruption.
Inheritance	Inherits	Child absorbs accessible Parent behaviors and structures.	Eliminates code duplication across logically related models.
Overloading	Overloads	Same method name, distinct parameter signatures within a class.	Compile-time polymorphism; simplifies user interfaces.
Overriding	Overrides	Child intercepts and replaces base method implementation.	Runtime polymorphism; dynamic execution based on concrete types.
Virtual Method	Overridable	Base provides complete code; Child optionally overrides.	Establishes standard fallback logic with customization hooks.
Abstract Class	MustInherit	Cannot be instantiated; serves strictly as a base template.	Defines rigid contracts and unified taxonomies for subsystems.
Abstract Method	MustOverride	Signature only; forces concrete derived classes to implement logic.	Guarantees downstream polymorphic consistency.

Unit 4: Summary and Core Comparisons

The following reference table categorizes the core controls examined in this unit, contrasting their structural purposes and key event triggers.

Control Name	Primary Interface Function	Distinctive Property	Core Action Event
Form	Serves as the base container window for all child controls.	FormBorderStyle	Load / FormClosing
Button	Triggers definitive processing logic execution.	FlatStyle	Click

TextBox	Captures raw alphanumeric data input from the user.	Multiline / PasswordChar	TextChanged
Label	Renders read-only UI descriptions and titles.	AutoSize	None typically required
CheckBox	Captures independent, non-exclusive boolean options.	Checked	CheckedChanged
RadioButton	Enforces mutually exclusive selection within a group.	Checked	CheckedChanged
ListBox	Displays an open array of persistent selectable items.	SelectionMode	SelectedIndexChanged
ComboBox	Space-saving drop-down menu for single-item selection.	DropDownStyle	SelectedIndexChanged

Unit 5: Summary and Core Comparisons

The following table contrasts the defensive programming concepts, structural targets, and operational behaviors examined throughout this unit.

Concept / Mechanism	Target Focus	Processing Overhead	Primary Architectural Objective
Syntax Error	Source code grammar validation.	Zero runtime overhead (halts compilation).	Enforces strictly valid language structure prior to build.
Runtime Error	Dynamic operational execution limits.	High (triggers process termination if unhandled).	Signals environmental, hardware, or unmanaged integration failures.
Logic Error	Algorithmic and semantic accuracy.	Variable (causes silent downstream corruption).	Requires human correction of logical processing rules.

Try...Catch...Finally	Reactive Exception Interception.	Moderate to High memory cost during stack trace generation.	Prevents application crashes and guarantees resource cleanup.
UI Validation	Proactive input data inspection.	Extremely Low local processing overhead.	Intercepts invalid input instantly before processing cycles are wasted.
Validating Event	Single control focus boundary.	Low.	Traps input focus locally until valid formatting is achieved.
ErrorProvider	Visual interface user feedback.	Minimal interface rendering cost.	Guides users to input corrections without blocking UI interactivity.

Unit 6: Summary and Core Comparisons

The following table reviews the structural differences between desktop interface styles and basic stream handlers examined in this unit.

Concept / Component	Architectural Behavior	Primary Advantage	Real-World Limitation
SDI (Single Document)	Distinct top-level container windows per document.	Ideal for side-by-side comparison across modern multi-monitor setups.	Causes taskbar overload and desktop clutter when managing multiple files.
MDI (Multiple Document)	Child document windows encapsulated inside one Parent frame.	Organizes highly unified workflows under shared main menus and toolbars.	Child views are completely restricted from leaving the primary monitor boundary.
System.IO Namespace	Manages OS folder structures and raw I/O pipelines.	Unifies direct file operations through high-level classes.	Operations block thread execution if storage reads/writes are exceptionally slow.
StreamWriter	Translates strings into byte streams	Simplifies writing formatted text files	Unhandled exceptions prior to .Close() can

	pushed to local storage.	without manual byte conversions.	leave file locks permanently active.
StreamReader	Decodes underlying byte streams into standard string output.	Efficient sequential line-by-line or whole-document text extraction.	Loading massive files via ReadToEnd() risks out-of-memory application crashes.

Unit 8: Summary and Core Comparisons

The following matrix contrasts the architectural foundations of desktop components, legacy integration standards, and distributed web paradigms discussed throughout this unit.

Technology / Construct	Execution Environment	Primary Enterprise Purpose	Core Structural Challenge
Component Object Model (COM)	Unmanaged OS Memory space.	Enables cross-language component integration at the binary level.	Relies on manual reference counting; susceptible to "DLL Hell" registration conflicts.
COM Interop (RCW / CCW)	Bridge between Managed CLR and Unmanaged memory.	Allows seamless migration and backward compatibility for legacy software.	Introduces data marshalling execution overhead across memory boundaries.
User Control	Managed WinForms Desktop Container.	Encapsulates multiple standard UI tools into a single reusable layout.	Requires explicit programming of custom properties and events to expose internal configurations.
Extended Custom Control	Managed WinForms Desktop Container.	Overrides deep visual rendering or operational logic of a base class.	Higher complexity curve; requires direct interaction with low-level drawing engines.
ASP.NET Web Application	Centralized IIS Web Server.	Delivers zero-install, globally accessible applications via web browsers.	Must constantly compensate for the underlying stateless nature of the HTTP protocol.

Session State	Server RAM allocation.	Securely tracks continuous data specific to an individual web user.	Consumes server memory resources proportional to concurrent site traffic volume.
----------------------	------------------------	---	--